



Aula 08 - React

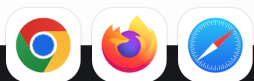
▼ Tipo	Aula
▼ Bimestre	1

React

- React é uma biblioteca JavaScript para construção de interfaces de usuário.
- React é usado para construir aplicativos de página única.
- React nos permite criar componentes de UI reutilizáveis.
- React, às vezes chamado de framework JavaScript frontend, é uma biblioteca JavaScript criada pelo Facebook.
- Ele permite que você junte componentes, mas não prescreve como fazer roteamento e busca de dados.
 - Para construir um aplicativo inteiro com React, recomendamos um framework React full stack como [Next.js](#) ou [Remix](#).

Use o melhor de cada plataforma

As pessoas adoram aplicativos da Web e nativos por diferentes motivos. O React permite criar aplicativos da web e aplicativos nativos usando as mesmas habilidades. Ele se baseia nos pontos fortes exclusivos de cada plataforma para permitir que suas interfaces pareçam perfeitas em todas as plataformas.



Comece um novo projeto React

Se você deseja construir um novo aplicativo ou site totalmente com React, recomendamos escolher uma das estruturas populares na comunidade com tecnologia React.

As estruturas fornecem recursos que a maioria dos aplicativos e sites eventualmente precisa, incluindo roteamento, busca de dados e geração de HTML.

Observação

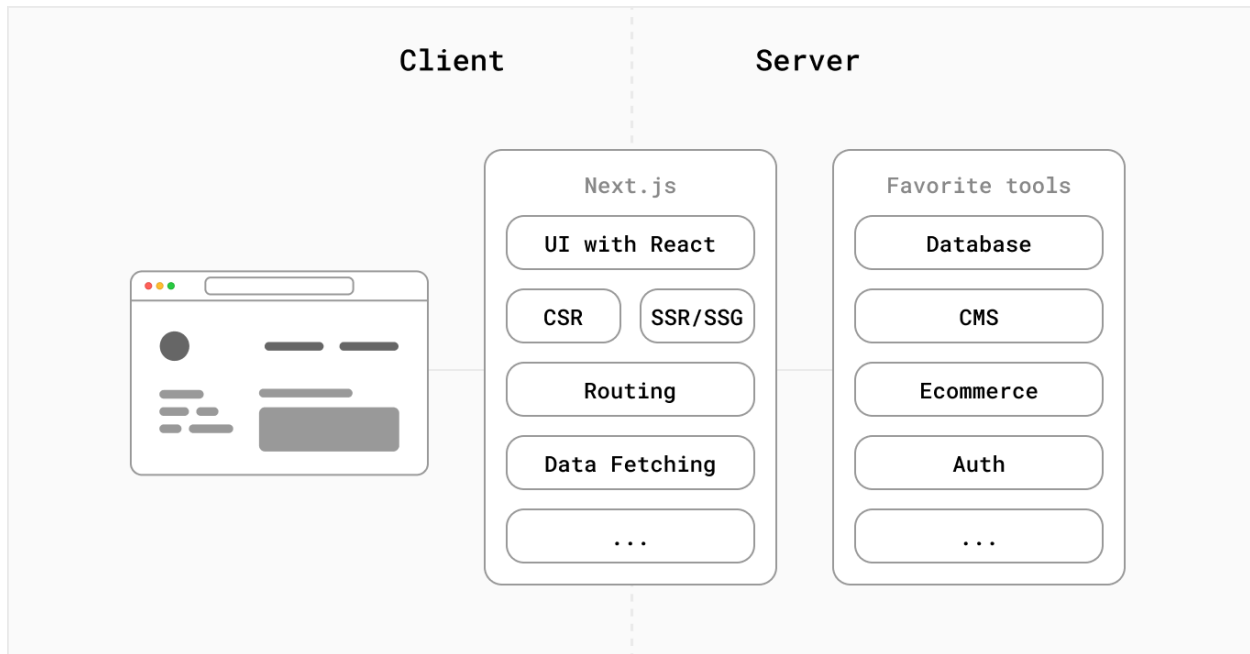
Você precisa instalar o Node.js para desenvolvimento local. Você *também* pode optar por usar o Node.js na produção, mas não é necessário. Muitas estruturas React suportam exportação para uma pasta HTML/CSS/JS estática.

Next.js

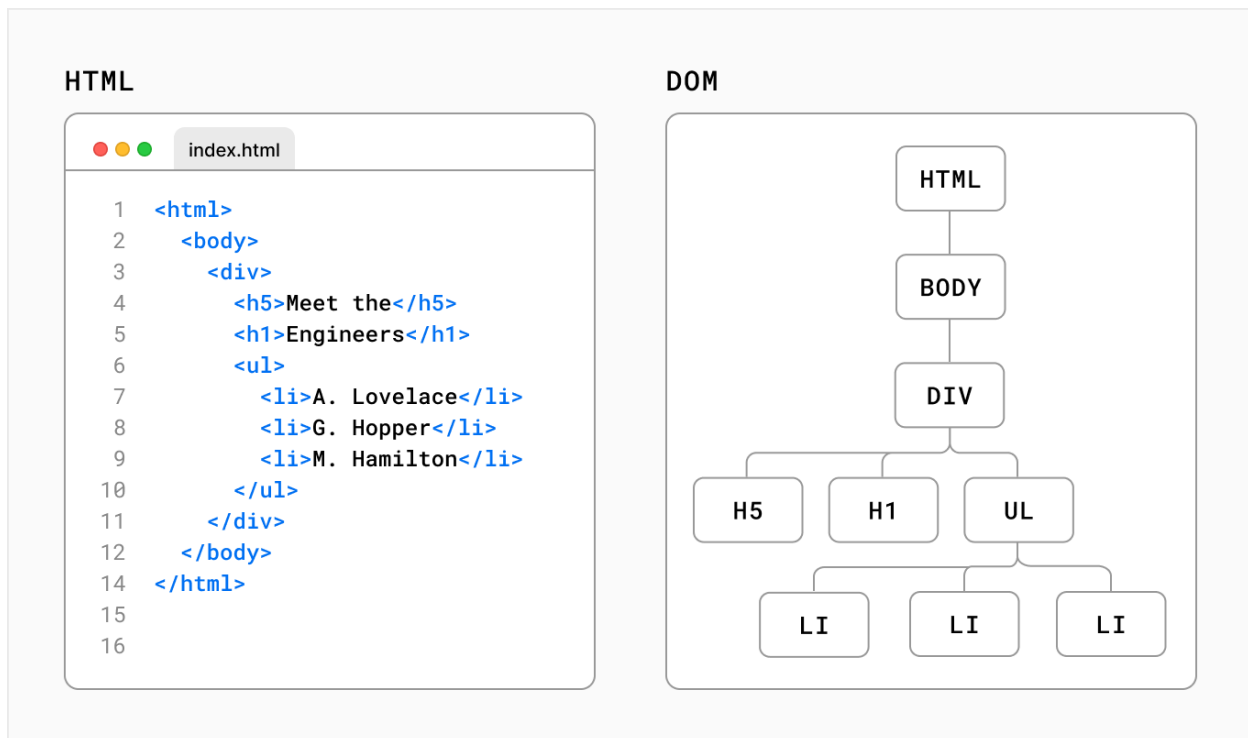
Next.js é uma estrutura React full-stack.

É versátil e permite criar aplicativos React de qualquer tamanho, desde um blog quase estático até um aplicativo dinâmico complexo.

Lida com as ferramentas e configurações necessárias para React e fornece estrutura, recursos e otimizações adicionais para seu aplicativo.



Quando um usuário visita uma página da web, o servidor retorna um arquivo HTML para o navegador que pode ter a seguinte aparência:

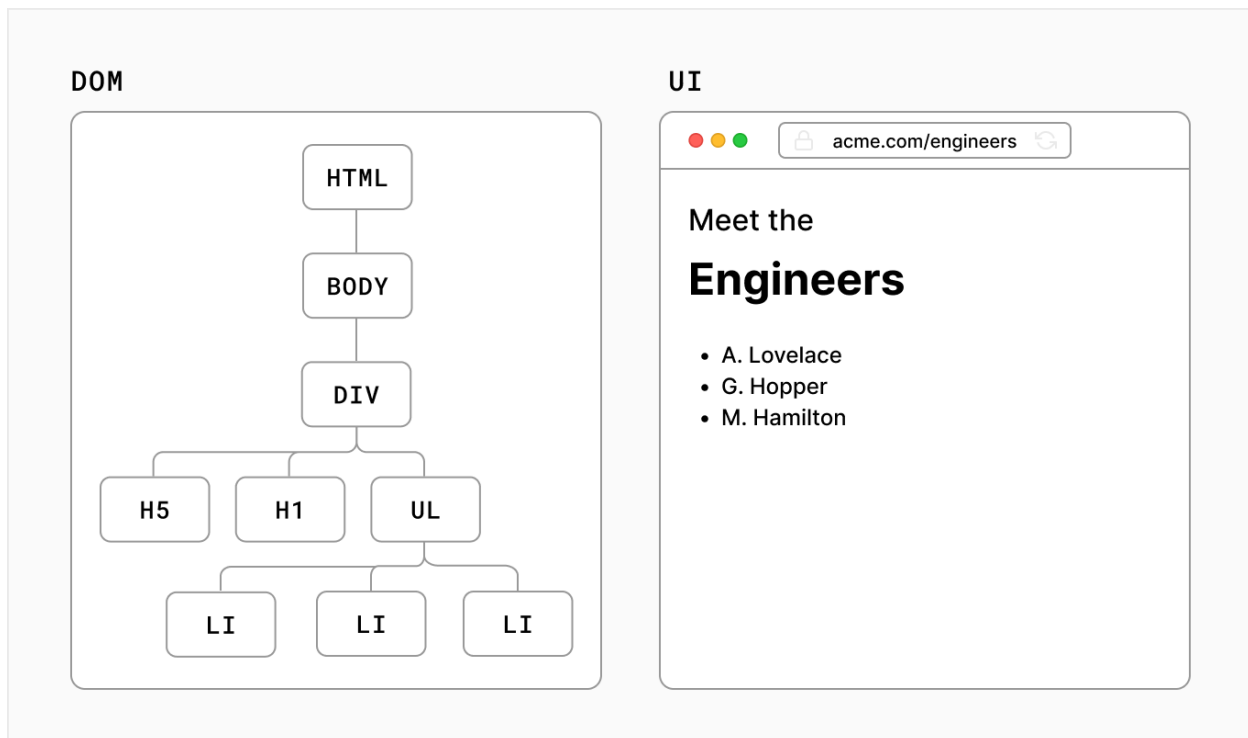


O navegador então lê o HTML e constrói o Document Object Model (DOM).

O que é o DOM?

O DOM é uma representação de objeto dos elementos HTML.

Ele atua como uma ponte entre o seu código e a interface do usuário e possui uma estrutura semelhante a uma árvore com relacionamentos pai e filho.



Você pode usar métodos DOM e uma linguagem de programação, como JavaScript, para ouvir eventos do usuário e manipular o DOM selecionando, adicionando, atualizando e excluindo elementos específicos na interface do usuário.

A manipulação do DOM permite não apenas direcionar elementos específicos, mas também alterar seu estilo e conteúdo.

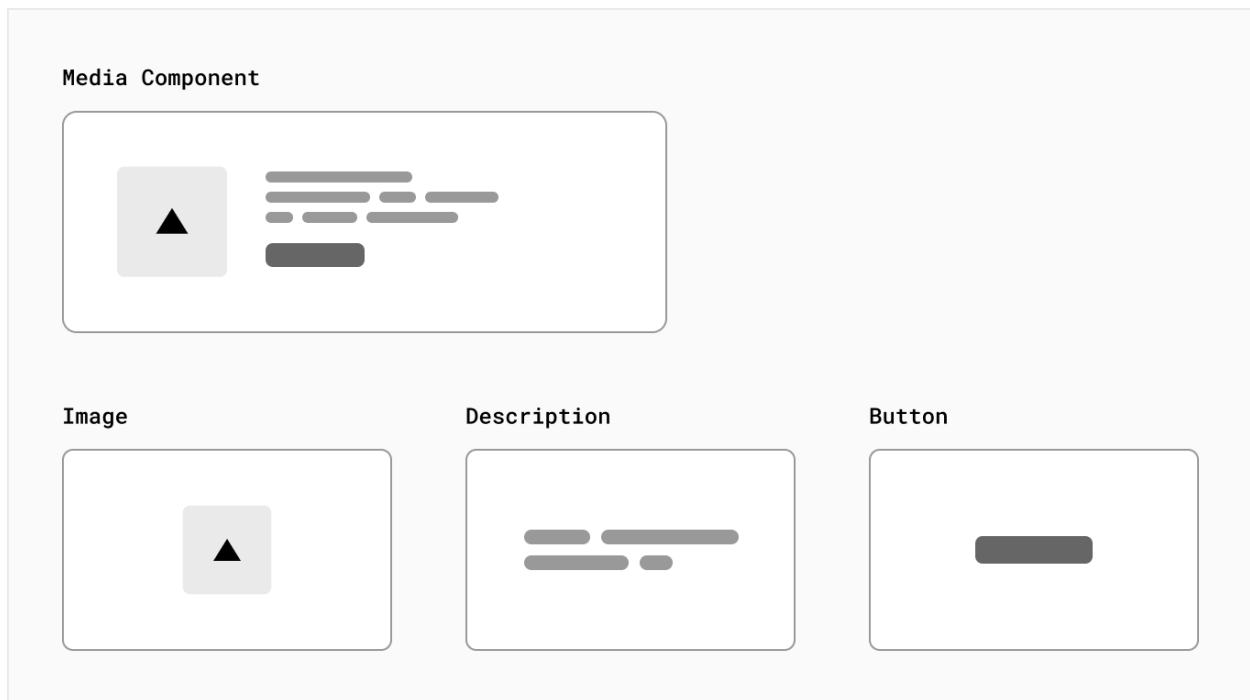
Construindo UI com Componentes

As interfaces de usuário podem ser divididas em blocos de construção menores chamados **componentes**.

Os componentes permitem que você crie trechos de código independentes e reutilizáveis.

Se você pensa nos componentes como **peças de LEGO**, pode pegar esses blocos individuais e combiná-los para formar estruturas maiores.

Se precisar atualizar uma parte da IU, você pode atualizar o componente ou bloco específico.



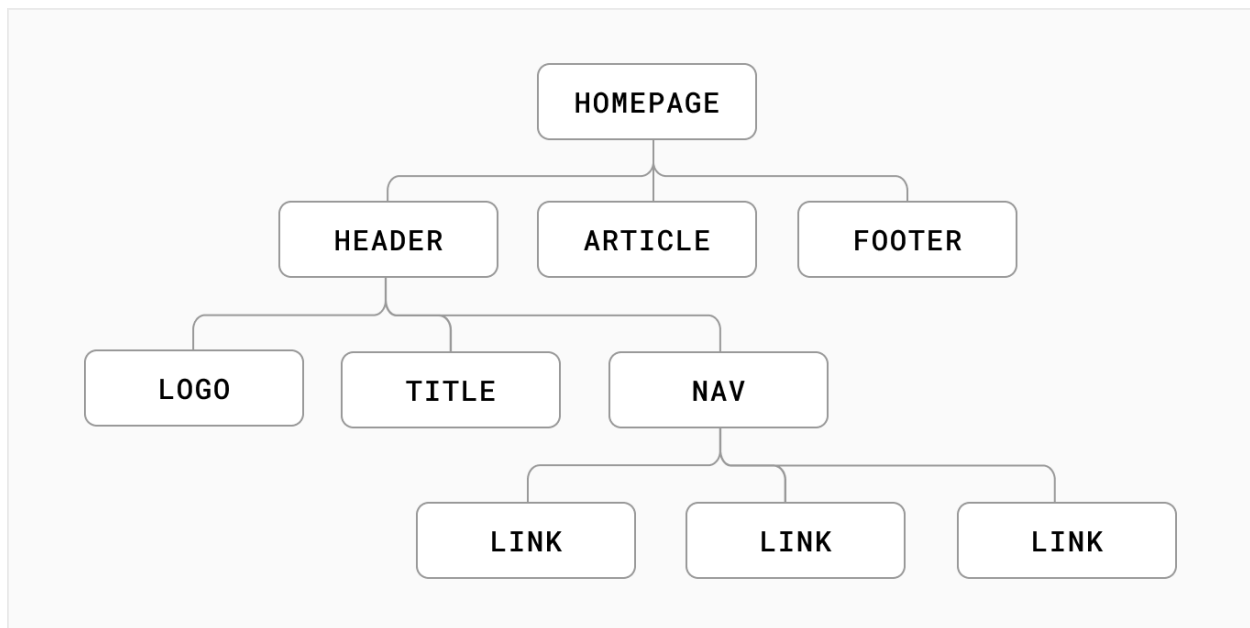
Essa modularidade permite que seu código seja mais sustentável à medida que cresce, porque você pode facilmente adicionar, atualizar e excluir componentes sem tocar no restante do nosso aplicativo.

O bom dos componentes React é que eles são apenas JavaScript.

Vamos ver como você pode escrever um componente React, da perspectiva JavaScript:

```
function Header() {  
  return <h1>Develop. Preview. Ship. 🚀</h1>;  
}  
  
function HomePage() {  
  return (  
    <div>  
      <Header />  
    </div>  
  );  
}
```

Você pode continuar aninhando componentes React dessa forma para formar árvores de componentes.



Por exemplo, seu componente de nível superior `HomePage` pode conter a `Header`, um `Article` e o `Footer` Component.

E cada um desses componentes poderia, por sua vez, ter seus próprios componentes filhos e assim por diante.

Por exemplo, o `Header` componente pode conter um `Logo` e `Title` um componente `Navigation`.

Primeiro Projeto

```
npx create-next-app@latest aula01
```

```
→ npx create-next-app@latest aula01
Need to install the following packages:
  create-next-app@13.5.6
Ok to proceed? (y) y
✓ Would you like to use TypeScript? ... No / Yes
✓ Would you like to use ESLint? ... No / Yes
✓ Would you like to use Tailwind CSS? ... No / Yes
✓ Would you like to use `src/` directory? ... No / Yes
✓ Would you like to use App Router? (recommended) ... No / Yes
? Would you like to customize the default import alias (@/*)? > No /
✓ Would you like to customize the default import alias (@/*)? ... No /
Yes
```

```
cd aula01
code .
npm run dev
```

pages/index.tsx

```
export default function Home() {
  return (
    <div>
      <h1 className="text-3xl font-bold underline">Hello world!</h1>
    </div>
  )
}
```

pages/index.tsx

```
function MeuBotao() {
  return (
    <button className="bg-indigo-50 text-cyan-800 p-1">
      Eu sou um botão
    </button>
  );
}

export default function Home() {
  return (
    <div>
      <h1 className="text-3xl font-bold underline">Hello world!</h1>
      <MeuBotao />
    </div>
  );
}
```

pages/index.tsx

```
function Header() {
  return <h1 className="text-3xl m-2"> Cabeçalho da página 🚀</h1>;
}
```

```
export default function Home() {
  return (
    <div>
      <Header />
    </div>
  );
}
```

No seu `Home` componente, você pode passar uma `title` propriedade personalizada para o `Header` componente, assim como passaria atributos HTML:

```
export default function Home() {
  return (
    <div>
      <Header title="React 🚀" />
    </div>
  );
}
```

E `Header`, o componente filho, pode aceitar esses adereços como seu primeiro **parâmetro de função**:

```
function Header(props) {
  console.log(props)
  console.log(props.title)
  return <h1 className="text-3xl m-2"> Cabeçalho da página 🚀</h1>;
}
```

Como props é um objeto, você pode usar **a desestruturação de objetos** para nomear explicitamente os valores de props dentro dos parâmetros de sua função:

```
function Header({ title }) {
  return <h1 className="text-3xl m-2"> Cabeçalho da página 🚀</h1>;
}
```

Para usar a variável que você definiu, você pode usar **chaves** `{ }`, uma sintaxe JSX especial que permite escrever JavaScript regular diretamente dentro de sua marcação JSX.

```
function Header({ title }) {
  console.log(title);
}
```

```
return <h1 className="text-3xl m-2"> Cabeçalho da página 🚀: {title}</h1>;  
}
```

Iterando através de listas

É comum ter dados que você precisa mostrar em forma de lista.

Você pode usar métodos de array para manipular seus dados e gerar elementos de UI com estilo idêntico, mas contendo informações diferentes.

Você pode então usar o `array.map()` método para iterar sobre o array e usar uma **função de seta** para mapear um nome para um item da lista:

```
function Header({ title }) {  
  console.log(title);  
  return <h1 className="text-3xl m-2"> Cabeçalho da página 🚀: {title}</h1>;  
}  
  
export default function Home() {  
  const names = ["Ada Lovelace", "Grace Hopper", "Margaret Hamilton"];  
  
  return (  
    <div>  
      <Header title="React 🚀" />  
      <ul className="m1-2">  
        {names.map((name) => (  
          <li key={name}>{name}</li>  
        ))}  
      </ul>  
    </div>  
  );  
}
```

Observe como você usou chaves para entrar e sair do terreno "JavaScript" e "JSX".

O React precisa de algo para identificar exclusivamente os itens em um array para saber quais elementos atualizar no DOM.

Você pode usar os nomes por enquanto, já que eles são únicos no momento, mas é recomendado usar algo com garantia de exclusividade, como um ID de item.

Adicionando Interatividade com Estado

Vamos explorar como o React nos ajuda a adicionar interatividade com **manipuladores de estado** e eventos .

Por exemplo, vamos criar um botão curtir dentro do seu componente `Home` .

- Primeiro, adicione um elemento de botão dentro da `return()` instrução.
- Para fazer o botão fazer algo quando clicado, você pode usar o evento `onClick`.
 - O `onClick` evento é um dos muitos eventos possíveis que você pode usar para responder à interação do usuário.
- Você pode definir uma função para "tratar" eventos sempre que eles forem acionados.
 - Crie uma função antes da instrução `return` chamada `handleClick()`:

```
export default function Home() {
  const names = ["Ada Lovelace", "Grace Hopper", "Margaret Hamilton"];

  function handleClick() {
    console.log("Incrementa os likes")
  }

  return (
    <div>
      <Header title="React 🚀" />
      <ul className="ml-2">
        {names.map((name) => (
          <li key={name}>{name}</li>
        ))}
      </ul>
      <button className="m-2 bg-blue-500 hover:bg-blue-700 text-white font-bold py-2 px-4 rounded"
        onClick={handleClick}>
        >Like</button>
    </div>
  );
}
```

Estado e hooks

React possui um conjunto de funções chamadas hooks.

Os ganchos permitem adicionar lógica adicional, como **estado**, aos seus componentes.

Você pode pensar no estado como qualquer informação em sua IU que muda ao longo do tempo, geralmente acionada pela interação do usuário.

Você pode *usar o estado* para armazenar e aumentar o número de vezes que um usuário clicou no botão Curtir.

- O React hooks para gerenciar o estado é chamado: `useState()`

- `useState()` retorna um array, e você pode acessar e usar esses valores de array dentro do seu componente usando **desestruturação de array**.
- O primeiro item da matriz é o state `value`, que você pode nomear qualquer coisa.
 - Recomenda-se nomeá-lo com algo descritivo.
- O segundo item da matriz é uma função para `update` o valor.
 - Você pode nomear qualquer função de atualização, mas é comum prefixá-la com `set` seguido do nome da variável de estado que você está atualizando.
- Você também pode aproveitar para somar o valor inicial do seu `likes` estado: zero.

```
function Home() {
  const [likes, setLikes] = React.useState(0);

  // ...
}
```

Você pode chamar sua função atualizadora de estado, `setLikes` em seu `Home` componente, vamos adicioná-la dentro da `handleClick()` função que você definiu anteriormente:

```
export default function Home() {
  const [likes, setLikes] = React.useState(0);

  function handleClick() {
    setLikes(likes + 1);
  }

  const names = ["Ada Lovelace", "Grace Hopper", "Margaret Hamilton"];

  return (
    <div>
      <Header title="React 🚀" />
      <ul className="m1-2">
        {names.map((name) => (
          <li key={name}>{name}</li>
        ))}
      </ul>
      <button
        className="m-2 bg-blue-500 hover:bg-blue-700 text-white font-bold py-2 px-4 rounded"
        onClick={handleClick}
      >
        Likes ({likes})
      </button>
    </div>
  )
}
```

```
};  
}
```

Páginas em Next.js

Em Next.js, uma página é um componente React exportado de um arquivo no diretório `pages` .

As páginas são associadas a uma rota com base no **nome do arquivo** .

Por exemplo, em desenvolvimento:

- `pages/index.js/` está associado à rota.
- `pages/posts/first-post.js/posts/first-post` está associado à rota.

Crie uma nova página

Crie o `posts` diretório em `pages` .

Crie um arquivo chamado `primeiro-post.tsx` dentro do diretório `posts` com o seguinte conteúdo:

```
export default function PrimeiroPost() {  
  return <h1>Primeiro Post</h1>;  
}
```

O componente pode ter qualquer nome, mas você deve exportá-lo como uma `default` exportação.

Agora, certifique-se de que o servidor de desenvolvimento esteja em execução e visite <http://localhost:3000/posts/primeiro-post> .

É assim que você pode criar páginas diferentes em Next.js.

Basta criar um arquivo JS no `pages` diretório e o caminho para o arquivo se tornará o caminho do URL.

De certa forma, isso é semelhante a construir sites usando arquivos HTML ou PHP. Em vez de escrever HTML você escreve JSX e usa componentes React.

Componente de link

Ao vincular páginas em sites, você usa a `<a>` tag HTML.

No Next.js, você pode usar o `Link` componente `next/link` para vincular as páginas do seu aplicativo.

`<Link>` permite que você faça navegação no lado do cliente e aceita acessórios que oferecem melhor controle sobre o comportamento da navegação.

Usando `<Link>`

- Primeiro, abra `pages/index.tsx` e importe o `Link` componente `next/link`

```
import Link from 'next/link';
```

- Use o componente:

index.tsx

```
import React from "react";
import Link from 'next/link';

export default function Home() {

  return (
    <div>
      <h1 className=" m-2 text-3xl">
        Ler o <Link className="text-blue-600 hover:underline" href="/posts/primeiro-post">primei
        ro Post</Link>
      </h1>
    </div>
  );
}
```

primeiro-post.tsx

```
import React from "react";
import Link from 'next/link';

export default function PrimeiroPost() {
  return (
    <div className="m-4">
      <h1 className="text-3xl">
        Primeiro Post
      </h1>
      <p>Texto do post</p>
      <h2>
        <Link className="text-blue-600 hover:underline" href="/">Voltar</Link>
      </h2>
    </div>
  );
}
```

```
);  
}
```

Fontes:

Quick Start – React

The library for web and native user interfaces

 <https://react.dev/learn>



Learn React

REACT.DEV/LEARN

Next.js by Vercel - The React Framework

Production grade React applications that scale. The world's leading companies use Next.js by Vercel to build static and dynamic websites and web applications.

 <https://nextjs.org/>



The React Framework for the Web